

Semi-Automatic Creation of System Models based on SysML Model Libraries

Yousef Hooshmand¹, Dmytro Adamenko², Steffen Kunnen³, Peter Köhler⁴

¹ Researcher, Universität Duisburg-Essen, Lotharstraße 1, 47057 Duisburg,
yousef.hooshmand@uni-due.de

² Researcher, Universität Duisburg-Essen, Lotharstraße 1, 47057 Duisburg,
dmytro.adamenko@uni-due.de

³ Researcher, Universität Duisburg-Essen, Lotharstraße 1, 47057 Duisburg,
steffen.kunnen@uni-due.de

⁴ Professor, Universität Duisburg-Essen, Lotharstraße 1, 47057 Duisburg,
peter.koehler@uni-due.de

Abstract. The increasing complexity of industrial plants is reflected in the system development and system modelling process. However, the complexity of the system model increases generally over proportionally. This makes the development, the administration and above all the reuse of the system models very difficult. This paper presents a method for semi-automatic creation of industrial plant models by using model libraries. For this purpose, an integrated IT infrastructure and an ontology based model library has been set up. The library is developed based on SysML notation, covers all RFLP abstraction levels and links the elements at all levels, which enables bidirectional impact analysis. The library is project-neutral and forms the basis for building project-specific system models based on the developed IT infrastructure. This increases, inter alia, the reuse of knowledge in companies. In addition to the industrial plant sector, the method presented is applicable to other domains.

1 Introduction

Companies with individualized products/ solutions try to meet the different customer requirements by creating and offering customer-specific variants, which serve as an important differentiating feature for improving the competitive position. However, this drastically increases the variety and the complexity of the product range, which is associated with increased effort and increases costs in almost all areas of the company. [Ho15] Thus, through various measures, the companies try to reduce or cope with the constantly increasing complexity. As the diversity of variants is one of the main sources of corporate complexity, variant management and complexity management are often used synonymously [Sch14].

The complexity drivers can be divided into exogenous and endogenous complexity drivers. The endogenous or internal complexity drivers are very strongly influenced and driven by exogenous or external complexity drivers. One of the strategic goals of variant management is indeed to maintain external diversity while simultaneously reducing internal diversity [Ho15]. The increasing complexity and variety of products induce

usually an over proportional increase of the complexity of system models and system development processes. Systems engineering (SE) and model-based systems engineering (MBSE) approaches are usually effective answers to these issues. The efficiency of these approaches and the methodologies developed for them is not however satisfactory.

Model-centered approaches are particularly capable of formally defining and describing systems that are necessary, inter alia, to enable system-thinking perspectives and the automatic processing of interrelationships between system elements as well as the impact analysis of changes. The Model-Based Systems Engineering (MBSE) approach has therefore gained increasing attention during the last decade and is going to be one of the cornerstones of the next industrial revolution called "Industry 4.0". [HAKK17]

INCOSE defines MBSE as "*the formalized application of modeling to support system requirements, design, analysis, verification and validation activities beginning in the conceptual design phase and continuing throughout development and later life cycle phases. MBSE is part of a long-term trend toward model-centric approaches adopted by other engineering disciplines, including mechanical, electrical and software*" [INC07]. The basic idea of MBSE is therefore to formalize the system description and to link the relevant information - required for the generation of various artifacts during the system development - in a system model [KS16].

In an MBSE approach the system model is set as a primary artefact of the systems engineering process, which helps to improve product quality, to improve reuse of system modeling artifacts, to improve communication between system developers and teams, to reduce time and costs for the integration and testing of the system and finally to reduce cost, schedule and risks in fielding a system [Seb16]. This helps consequently to increase the transparency of development steps as well as the traceability of changes and to improve the handling of complexity [KH16].

Three key enablers or aspects are needed to leverage MBSE's potential: *modeling languages, methods* and *tools*. For an effective and efficient implementation of MBSE approach all three aspects need to be considered at the same time. The paper's contributions to the crucial enablers are as following. These are mainly based on the author's previous publication "An approach for holistic model-based engineering of industrial plants" (see [HAKK17]).

1. *Modeling language*: The OMG Systems Modeling Language (OMG SysML), a formalized graphical modeling language, has been used as the main building block of this work. First of all, SysML has been enriched with various ontologies that enhances the semantic of the language. This, inter alia, increases the semantic traceability of changes and makes an effective impact analysis possible. Furthermore, a model library based on SysML notation has been developed, which makes working with the language significantly more efficient and also more attractive to use.

2. *Methods*: The MBSE method developed in this work is based on RFLP abstraction levels (i.e. Requirements (R), Functional (F), Logical (L) and Physical (P) levels). It helps to create a holistic system model as a single source of truth. The method uses both project-neutral model libraries as well as a reverse engineering approach for utilizing

available domain models. This ensures, inter alia, the traceability along the development processes as well as increases the efficiency in searching possible solutions in different life cycle stages.

3. *Tools*: To support the presented method, an integrated IT infrastructure has been developed. This includes, inter alia, a REST web service, a user interface, a model library and a few plug-ins for different tools. The developed infrastructure helps to make the model creation semi-automatic. Furthermore almost anyone even without a deep knowledge of SysML can work with the tools and contribute to the creation of the system model as a single source of truth. This increases drastically the efficiency and the effectivity of system model creation and makes it much more error-free.

Next sections describe the needed steps for the creation of the named model library as well as the integrated IT infrastructure.

2 Set-up of the model library using SysML

Considering that different industrial plants are increasingly using the same system elements only in different constellations and sequences, the creation of a model and variant library for system elements (stereo types «*requirements*», «*functional*», «*logical*» and «*physical*») is a first step that enables elements to be used across projects. This is particularly the case on a functional and logical levels and also partially on a physical and requirements levels. In addition, the definition of relationship ontologies between different levels can be partially project- and plant-independent. This serves, inter alia, to exploit synergy effects by formalizing the extensive knowledge scattered in various project teams and disciplines and making it available to everyone in the organization. This section describes the structure of a project-neutral model library and its use for project-specific model design.

The SysML notation has been used as a basis for building the model library in a central database. The model elements and their variants are described and linked at different RFLP levels (in particular function level, logical level and physical level). The elements in the library are then used later directly to build project-specific SysML models. The model library is built up in following steps.

2.1 Definition of elements

The elements at different levels can be defined separately from each other and subsequently linked. In this work the logical elements, which are mainly project neutral, have been defined first. The logical elements have been then used to define the elements at function level. Finally, the project-neutral elements are to be defined at the physical level and at the requirement level. In this work, the SysML ports and connectors are also defined as elements. Each element receives a unique ID number. The stereotypes that are used at RFLP levels are

- «Requirement», «Functional», «Logical» and «Physical».

Based on the defined ontologies, project-neutral structures and networks can be defined in the model library. The structures and networks are defined in particular at F and L level. Thus, for example, the structure or assembly of the element "pump" and its variant can be defined in a project-neutral manner. In addition, the interactions between elements on the same level can be defined and described as a network.

Figure 1 shows as an example the elements and the structure of a pump shown in a BDD diagram at the L level.

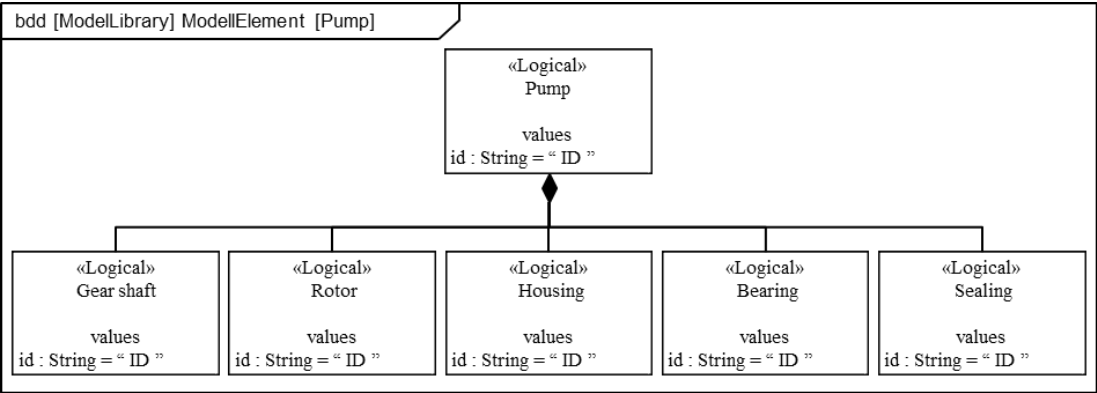


Figure 1. Structure of a pump at the L level (based on [Ho15])

Figure 2 shows the simplified functionality of a pipe cleaning system illustrated as a network in an IBD diagram at F level.

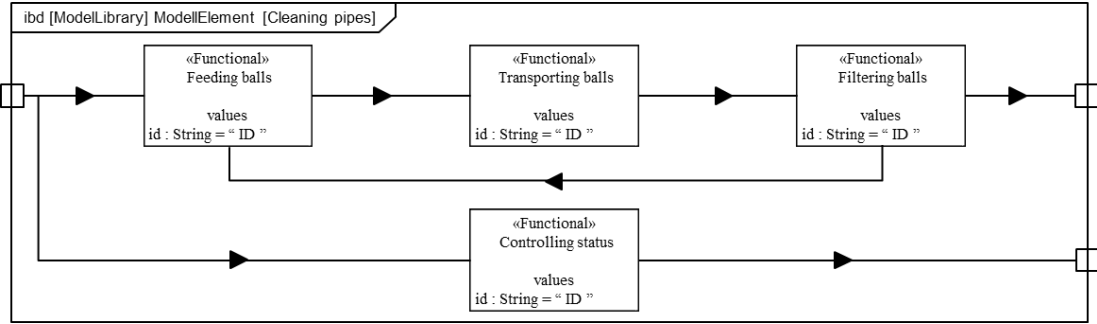


Figure 2. Pipe cleaning system shown in the model library at the F level

2.2 Definition of variants

For each element, first the characteristics, which are necessary for a more precise description and identification of the element, have to be defined. To standardize the definition and description of elements and variants, the desired characteristics and values are defined on the basis of eCl@ss. The characteristics and values therefore have a unique identification number. This ensures the comparability of elements/variants and enables inter alia a more efficient analysis. These characteristics are also used to define the required element variants. This creates a project-neutral variant library for each element. Similarly, the variants also receive a unique ID number. The relationship ontology between elements and variants at FLP levels is

- *«is.a.generalization.of»*.

Figure 3 shows the "Pump" element and its two variants. As shown, the "pump" element is at the logical level and the variants are therefore defined also at the logical level.

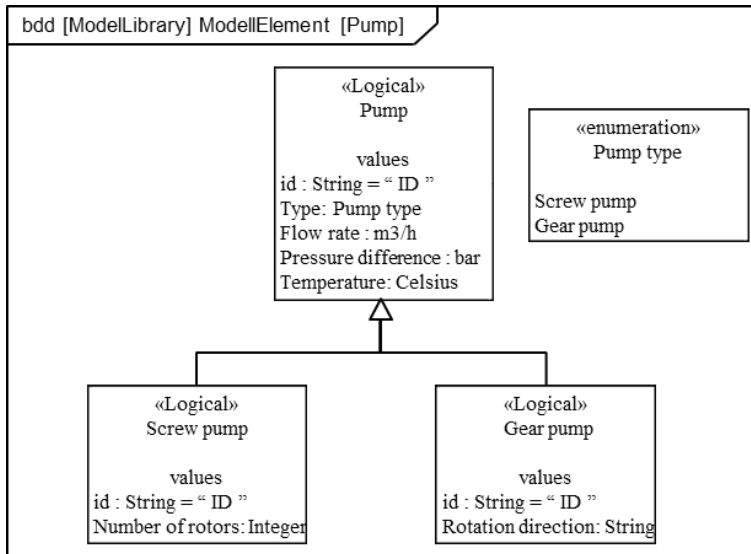


Figure 3. Pump features and variants at L level (based on [Ho15])

2.3 Ontology based linking of elements/ variants

Finally, the elements/variants are to be linked at the same RFLP levels as well as at different RFLP levels. This is particularly necessary to ensure traceability at all levels. The elements at the same level are linked with the relationship ontologies

- *«has.spatial.interaction.with»*, *«has.energy.flow.with»*, *«has.information.flow.with»*, *«has.material.flow.with»*, *«is.an.aggregation.of»*, *«is.a.composition.of»* and *«is.a.generalization.of»* for FLP-levels and

- «derive» and «trace» for the R level.

The elements at different RFLP levels are linked with the relationship ontologies

- «is.allocated.to» between F and L levels,
- «is.allocated.to» and «is.a.generalization.of» between L and P levels and
- «satisfy» between R and FLP levels.

Figure 4 shows the structure of the model library at RFLP levels. In this work only BDD, IBD and REQ diagrams are used to develop the model library. Selected ontologies are also defined to describe the relationships between elements. Additional diagrams and detailed ontologies can be added according to company requirements.

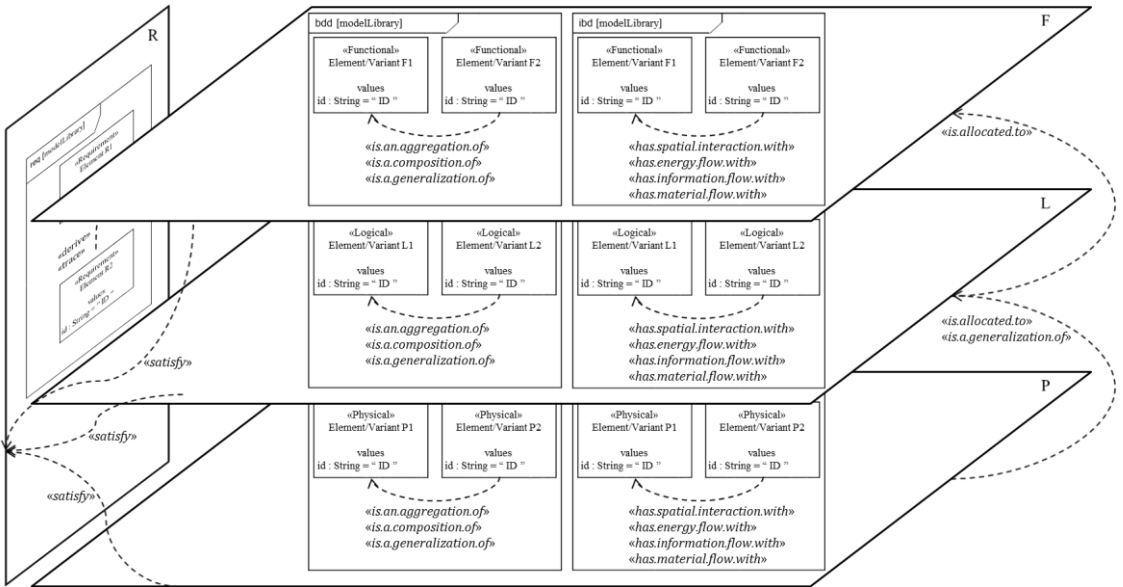


Figure 4. Structure of the model library at all RFLP levels (incl. diagrams and ontologies)

The project-neutral models can drastically accelerate the project-specific model structure and make it error-free. Ideally, the model library should contain all elements and their variants (especially at F and L levels) and describe them with uniform features and characteristics. This enables the generation of knowledge and increases its reuse. Thus, the knowledge gained from completed and ongoing projects flows into the model library and forms the knowledge base for future projects in the company. Similar to the simulation tool Modelica, the presented model library can enable cross-company reuse of standardized components.

The project-neutral structures and networks can usually only be implemented for subsystems. The entire system is often only project-specific to be defined, which is described in the next section.

3. Project specific creation of system models

The engineering of industrial plants or also of power plants takes place in different iterations. Based on stakeholder and system requirements, a procedural layout design is carried out - including, for instance, functional, thermal and process analysis. In the first iterations, both rough functional layouts (functional structures) and process layouts are created, which consist only of technology- and implementation-independent components (logical structures). The result of final iterations are fine-specified process layouts (physical structures). The results of the plant construction process are mainly documented in piping and instrumentation diagrams (P&ID), which are often created on the basis of various international standards. These P&IDs generated by domain engineers are used to successively enhance the system model in SysML at various levels. However, the creation and enhancement of the system model in SysML is not limited to P&IDs and other models and artifacts are used in further life cycle phases.

The P&IDs contain almost all the necessary information about the system elements of the plant and the relationships between these system elements. P&IDs created with professional process engineering tools are machine-readable and their information can be read and converted into SysML models. This allows the system architecture to be semi-automatically created and visualized in block definition diagrams (bdd) and internal block definition diagrams (ibd). A semi-manual task will be the assignment of project-specific requirements to the elements of the system architecture in order to ensure their traceability. The system model in SysML will be created based on the previously developed element and variant libraries (including components, ports and connectors) and the stereotypes defined in the SysML profile, which also include the ontologies of the last section.

The presented model library has been developed using SysML notation and forms a project-neutral knowledge base. The project-specific system models are then built semi-automatically using the model library. All element relevant information is included when the elements are pulled in from the library. Thus, the project-specific system model is supplemented with information such as element structure, element properties and possible interactions with elements on other levels directly when an element (e.g. pump) is drawn in from the library. This information no longer needs to be added separately and individually, which drastically reduces the likelihood of errors slipping in during model construction.

The project- and plant-independent definition of relationships between elements at different levels enables the analysis and comparison of different plant concepts and solutions based on functional and logical structures already during the first design iterations. The project-specific system models are created and managed in the PLM database and in the bidirectionally integrated SysML modeling tool Cameo Systems

Modeler. Figure 5 describe the integrated IT infrastructure and the interaction between various building blocks.

There are different possibilities for model building. Firstly, the systems engineer can pull the model elements one by one from the database, link them together and specify them more precisely if necessary. Thus the responsible engineer has the possibility to build up a system model even without deeper SysML knowledge and to successively enhance it. A simplified user interface is prepared for this and the project-specific system model is stored and managed on the database designed for this purpose after each operation. The second possibility for model building is to create the system model semi-automatically based on already developed domain models. Thus, for example, a P&ID model will be automatically read and the desired system model will be created in SysML based on the existing elements in the model library. In general both methods are used simultaneously to build the system models. The relationship between the system models and the domain models is often bidirectional and a system model can also be used as a basis for developing domain models. Figure 5 shows the (prototypically) developed infrastructure in an integrated landscape that enables holistic model-based engineering.

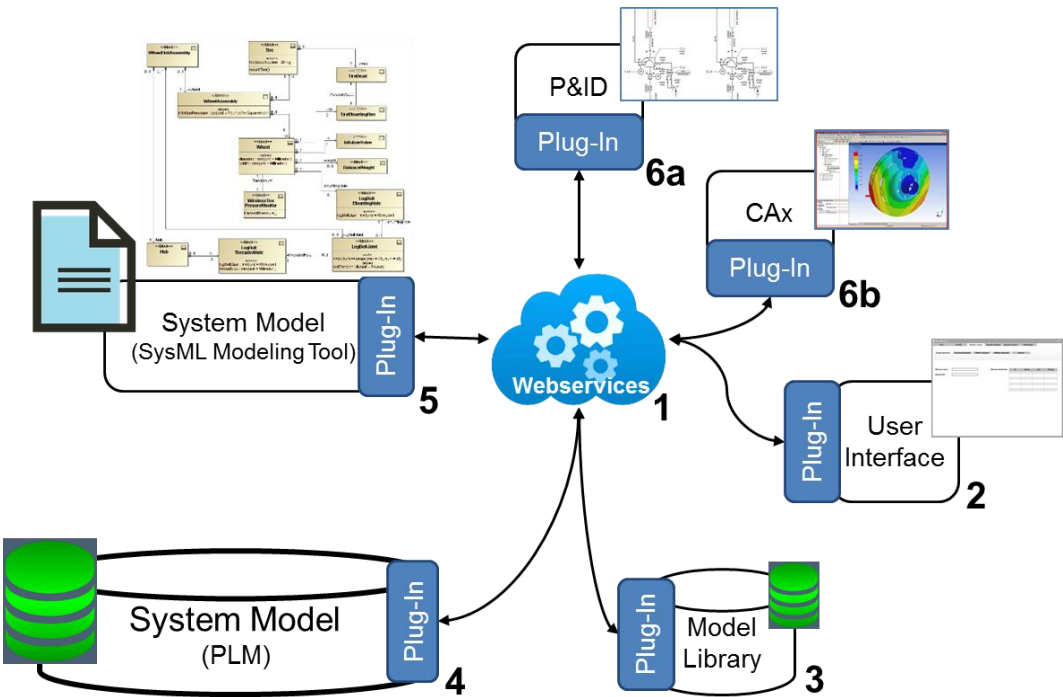


Figure 5. Integrated IT infrastructure to enable holistic model-based engineering (based on [HMK17])

The main modules of the infrastructure are illustrated in Figure 5 and are in use as following:

1. *Web services*: With the help of web services technology (Figure 5, No. 1) all system modules are integrated separately from one another, which makes a simple and flexible communication between these modules possible. A plug-in for data exchange has been developed for each module.

2. *User interface*: Simple user interfaces (Figure 5, No. 2) has been developed for both users and administrators. The administrators can maintain the models (elements, variants,...) in the model library (Figure 5, No. 3) and the users can build project-specific system models and manage them directly in the PLM system (Figure 5, No. 4) and in the SysML editor Cameo Systems Modeler (Figure 5, No. 5).

3. *Model library*: The presented model library (Figure 5, No. 3) is managed in a central database. All units of the model library are under version control to make the changes traceable. The model library can also be set up in a PLM system.

4. *System model in PLM*: The project-specific system models (Figure 5, No. 4) are placed under version control and managed in a PLM system. This is necessary to link the system models and the domain models and to enable the holistic model-based engineering approach.

5. *System model in SysML*: Using the SysML editor Cameo Systems Modeler (Figure 5, No. 5), both the models in the model library and the project-specific system models or its parts can be built automatically as SysML diagrams. The relationship is bi-directional and the changes in the SysML editor can be saved back. The interaction is controlled via the developed user interface (Figure 5, No. 2) and the plug-ins.

6. *Plug-Ins for domain models*: Using the plug-ins, the domain models (Figure 5, No. 6a and 6b) can be transformed into the system model. The relationship is partly bi-directional and the system model can be used as a basis for developing/ enhancing domain models (here e.g. P&ID). Interactions are also controlled via the developed user interface (Figure 5, No. 2) and the plug-ins.

The IT infrastructure presented in Figure 5 enables efficient reverse engineering for gaining knowledge from completed projects. Thus, the domain models of these projects can be semi-automatically transformed into an intelligent ontology-based system model. This makes the newly generated system models both comparable and easier to evaluate. This allows new, hidden insights to be gained from completed projects, which can be a valuable source of knowledge for future projects.

5. Summary

An approach for semi-automatic creation of system models based on an integrated IT infrastructure and a SysML-based model library has been presented. This has been done

using a SysML-based model library that maps the model elements and variants as well as the existing interactions using SysML notation. In order to increase the semantic traceability of changes during plant development and construction, an ontology has been developed that describes the relationships between system elements at different levels. This also forms the basis for the impact analysis of changes. The proposed approach increases traceability in the organization and enables a pragmatic implementation of MBSE methods in plant construction. However, it also applies to other sectors. The developed IT infrastructure enables the domain engineers with limited SysML knowledge to continue to work with their familiar tools and at the same time use the continually evolving system model as their single source of truth.

It's still necessary to examine the possibilities and requirements for an automatic extension of the project-neutral model library and the project-specific system model by other SysML diagrams. Furthermore, to increase the interoperability, both OSLC-based tool integration and the data formats of the created models and artifacts must be examined to ensure a loss-free and bidirectional model transformation.

Acknowledgement

The authors acknowledge the financial support by the *Federal Ministry for Economic Affairs and Energy* of Germany in the Project UMEK (Unterstützung von multidisziplinären Engineering-Prozessen im Kraftwerksbau).

Supported by:



Federal Ministry
for Economic Affairs
and Energy

on the basis of a decision
by the German Bundestag

References

- [HAKK17] Hooshmand, Y.; Adamenko, D.; Kunnen, S.; Köhler, P.: An approach for holistic model-based engineering of industrial plants. In: Proceedings of the 21st International Conference on Engineering Design (ICED17), Vol. 3: Product, Services and Systems Design, Vancouver, Canada, 21.-25.08.2017.
- [HMK17] Hooshmand, Y.; Manoharan, T., Köhler, P.: Unterstützung von multidisziplinären Engineering-Prozessen im Kraftwerksbau (UMEK). IN: Proceedings of the 32nd FDBR-Fachtagung Rohrleitungstechnik, Magdeburg, Germany, 28.-29.03.2017.

- [Ho15] Hooshmand, Y.: Transparenzerhöhung bei der Entwicklung von individualisierten Produkten in der Einzelfertigung, Verlag Dr.Hut, München 2015.
- [INC07] Systems Engineering Vision 2020, version 2.03, INCOSE 2007. Available at: http://oldsite.incose.org/ProductsPubs/pdf/SEVision2020_20071003_v2_03.pdf
- [KH16] Kleiner, S.; Husung, S.: Model Based Systems Engineering: Prinzipien, Anwendung, Beispiele, Erfahrung und Nutzen aus Praxissicht. In: Proceedings of Tag des Systems Engineering, Herzogenaurach, Germany, 25.-27.10.2016.
- [KS16] Kaufmann, U.; Schuler, R.: Systems Re-Engineering - ein Beitrag zur Integration von MBSE und PLM. In: Proceedings of Tag des Systems Engineering, Herzogenaurach, Germany, 25.-27.10.2016.
- [Sch14] Schoeneberg, K.-P.: Komplexität – Einführung in die Komplexitätsforschung und Herausforderungen für die Praxis. In: Schoeneberg, K.-P. (Hrsg.): Komplexitätsmanagement in Unternehmen. Herausforderungen im Umgang mit Dynamik, Unsicherheit und Komplexität meistern, Springer Gabler, Wiesbaden 2014.
- [Seb16] Transitioning Systems Engineering to a Model-based Discipline, Systems Engineering Body of Knowledge (SEBoK), Version 1.7, INCOESE 2016. Available at: www.sebokwiki.org/wiki/Transitioning_Systems_Engineering_to_a_Model-based_Discipline